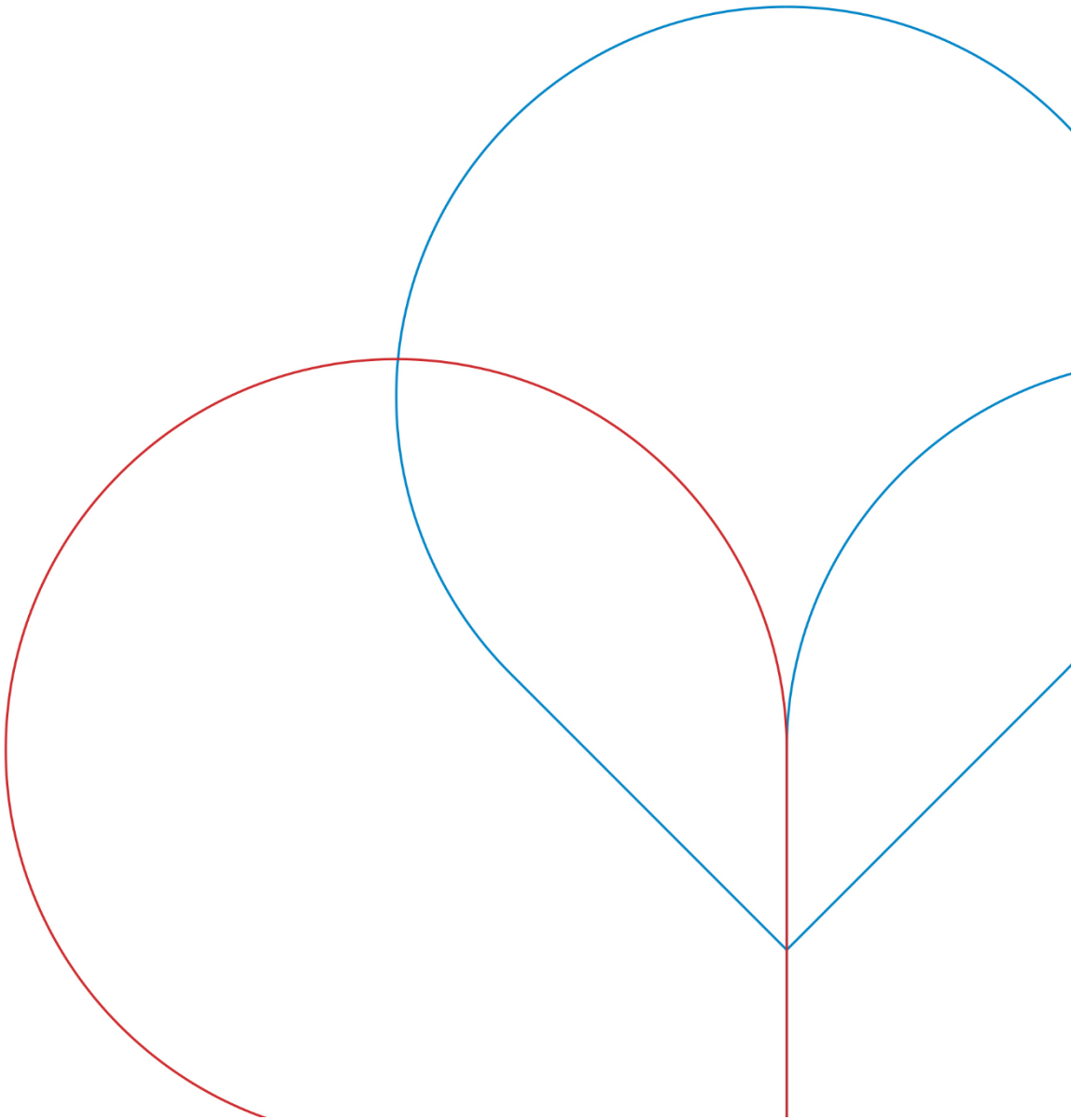
TruePoint 真点

真点高精度定位服务
集成软件开发指南 V4.1
（Linux 通用版）



版权声明

版权所有©真点科技（北京）有限公司2020-2026。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容，或以任何形式传播本文档内容。

注意

由于产品版本升级或其他原因，真点科技将定期对本文档内容进行更新或修正，但不会另行通知。若需获取最新版本文档，请联系真点科技相关人员。

本文档仅作为使用指导，文档内容可能包含技术上不准确或与产品功能及操作不相符的地方，以真点科技最终解释为准。本文档中的所有陈述、信息和建议等均不构成真点科技的任何明示或暗示的担保、保证。

商标声明

TruePoint 真点 和其他真点商标均为真点科技（北京）有限公司的商标。本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

真点科技（北京）有限公司

地址： 北京市海淀区丰贤东路 7 号北斗星通大厦北楼二层

网址： www.true-point.com

前言

目的

本文档为开发者提供真点科技高精度定位服务SDK (简称SDK)的使用说明。
该SDK适用于x86和arm架构的Linux系统。

适用对象

本文档适用于熟悉C语言的Linux开发者。

修订记录

版本号	修订记录	日期
R1.0	初版发布（SDK 4.0 阶段开发指南）	2026-03
R2.0	SDK 4.1 版开发指南修订： ① 增补快速入门（配置文件示例、默认配置、坐标框架说明）； ② 增补「设置配置文件路径」及 `tp_sdk_init` 检测说明、初始化流程图； ③ 各接口补充「接口常见错误码」，与 `tp_sdk_status_code.h` 对齐； ④ 更新「错误码」总表（含 6000～6999 平台码说明）； ⑤ 增补 `tp_send_gga` 流程与重连说明； ⑥ 更新术语表（如 x86）、开发须知与调用流程图	2026-06

术语与缩略语

序号	术语/缩略语	解释说明
1	PPP-RTK	Precise Point Positioning - Real Time Kinematic（精密单点定位-实时动态测量）
2	NRTK	Network Real-Time Kinematic Positioning（网络实时动态定位技术）
3	SDK	Software Development Kit（软件开发工具包）
4	ARM	Advanced RISC Machine（进阶精简指令集机器）
5	AutoSAR	Automotive Open System Architecture（汽车开放系统架构）
6	GGA	Global Positioning System Fix Data（全球定位系统定位信息）
7	RTCM	Radio Technical Commission for Maritime Services (海事无线电技术委员会)
8	IP	Internet Protocol（网际互连协议）
9	GCC	GNU Compiler Collection（GNU编译器套件）
10	MSM4	MSM4 是 RTCM 标准中定义的 MSM（Multiple Signal Message，多信号消息）系列中的一种消息类型，用于传输完整的 GNSS 伪距、载波相位和载噪比观测数据
11	TLS	Transport Layer Security(传输层加密)
12	x86	x86 泛指一系列基于 Intel 8086 且向后兼容的中央处理器指令集架构
13	TCP	Transmission Control Protocol（传输控制协议）
14	SHA256	Secure Hash Algorithm（安全散列算法）中的一种
15	NMEA 0183	NMEA 0183是美国国家海洋电子协会 （National Marine Electronics Association ）为海用电子设备制定的标准格式
16	C99	C99（正式名称ISO/IEC 9899:1999）是C编程语言的第二个官方标准，于1999年发布，由ISO/IEC正式认证，ANSI 随后于2000年3月采用
17	ISO	International Organization for standardization（国际标准化组织）
18	IEC	International Electrotechnical Commission（国际电工委员会）
19	GNU	GNU 是一个自由软件项目
20	ANSI	American National Standards Institute(美国国家标准学会)

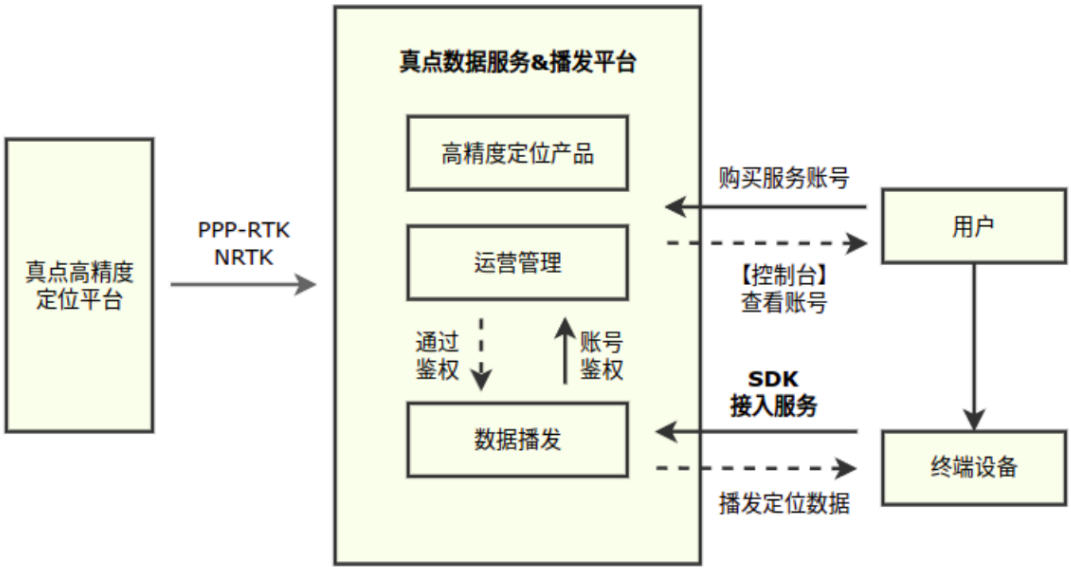
序号	术语/ 缩略语	解释说明
21	ID	Identifier（标识符）
22	MSM2	MSM2 是 RTCM 标准中定义的 MSM（Multiple Signal Message，多信号消息） 系列中的一种消息类型，专门用于传输紧凑型 GNSS 载波相位观测数据
23	SSL	Secure Sockets Layer（安全套接层）
24	DNS	Domain Name System（域名系统）
25	IPv4	Internet Protocol version 4（互联网协议的第四个版本）
26	IPv6	Internet Protocol Version 6（互联网协议第6版）

1 SDK简介

真点科技高精度定位服务集成软件开发包SDK(简称SDK) 是一款适用于Linux系统的库，适配于特定软件框架、操作系统、应用程序等。用户在终端设备上，通过网络连接到服务器，上传概略位置（GGA信息），用户自动获取差分数据，实现高精度定位的位置服务。

真点标准版SDK集成默认提供 **x86_64** 和 **aarch64** 编译链版本，其余嵌入式、ARM、AutoSAR等系统或架构均可定制，可通过联系与帮助联系真点技术支持或者通过控制台-工单-提交SDK申请，即可完成。

本SDK资源消耗小，是一款高效稳定，易用灵活的高精度定位软件开发包。



2 快速入门

本章主要介绍SDK的软硬件环境、账号获取方法、使用示例的编译及运行。

2.1 准备工作

2.1.1 软硬件环境

组件	配置	说明
代码编辑器	Visual Studio Code 1.106.3	常规C代码编辑器均可
示例开发环境	x86_64 GNU/Linux Ubuntu(24.04.1)	
GCC版本	GCC 13.3.0	使用C99标准

使用时需将真点科技方提供的库文件和头文件导入到相应的项目目录中。
如您需要定制其他编译版本，请提供指定交叉编译工具链或 SDK 开发包给技术支持或通过控制台提交工单-申请SDK，后续真点方开发验证之后提供。详情请联系真点科技的工作人员。

2.1.2 账号获取

开发者需先联系真点销售或市场部门相关人员购买高精度定位服务，获得账号。

2.2 示例运行

SDK开发包中提供使用示例及其编译的Makefile文件，运行make，生成可执行程序。可执行程序可以看到类似如下内容的帮助信息：

```
Usage: ./edge_app [OPTIONS]
Options:
  -u USER          用户名
  -p PASSWORD      密码
  -m MOUNTPOINT    挂载点
  -d DEVICE        串口设备路径（与 -g 二选一；未指定且未使用 -g 时默认为 /dev/ttyUSB0）
  -g GGA_STRING    模拟gga字符串（与 -d 二选一）
  -h               显示帮助信息
  --verbose LEVEL  测试程序日志控制，为0时仅显示测试程序必要信息
  --baud RATE      串口波特率，默认 115200
  --gga-interval SEC  GGA 上报间隔（秒），默认 1
  --send-gga-with-retry  可选择0或者1，0表示不进行重连，1进行重连，默认 0
  --block-time SEC  示例内 RTCM 超时判定基准（秒），默认 3
```

调用示例

```
./edge_app -u username -p password -m RTCM33GRCEJ -d /dev/ttyUSB0 --verbose 3 --baud 115200 --gga-interval 1 --send-gga-with-retry 1 --block-time 4
```

2.2.1 配置文件格式参考

```
#
#服务信息（必填）
#

# 账号
account=xxx

# 密码
password=xxx

# 挂载点
mount_point=RTCM33GRCEJ

#
# 服务器配置 (可选)
#

# 服务地址，格式为 hostname:ip，设置为 pnt.true-point.com:8801
server=pnt.true-point.com:8801

# 服务端的数据下发时间间隔，单位为秒，设置为1
rtcm_interval=1

# 网络传输数据最大等待时间，单位秒， 设置为3
block_time=3

# 坐标系，设置为1
coordinate_sys=1

# 设备ID， 设置为 devid-demo
device_id=device-demo

# 是否在发送GGA时进行重试，设置为0(不重试)
send_gga_with_retry=0

# 重试间隔时间，单位为秒，设置为10秒
retry_interval=10

# 重试次数
```

```
retry_number=3

# 认证类型，可选2002(私有协议)、2003(AKAS)
auth_type = 2002

# 是否支持GGA偏移计算，设置为1(支持)
support_gga_offset=1

# 服务类型，设置为1(NRTK)
server_type=1

# SDK 日志输出等级，可选项：verbose、debug、info、warning、error、none
log_level=debug

# Transport Layer Interface option: tcp or ssl
transport_protocol = tcp

# SSL option
server_ssl_verify = cert,cn
server_ssl_path = /etc/ssl/certs/ca-certificates.crt
```

注意
（1）配置文件中可以没有部分配置项，但"配置项="后面不能为空，不能为无效值（如-1），否则加载文件会报错。若希望设置为默认值，可以去掉配置项。
（2）若协议选择 ssl，但是端口配置成 8801，则检测配置时会报错。若没有配置成 8801，而是配置成其他端口，则配置检测时不会报错，后续通过接口配置参数同理。

sdk的配置优先级为用户调用接口配置 > 配置文件 > sdk默认配置，若存在重叠按照优先级，高优先级覆盖低优先级，若高优先级未配置，则使用紧邻的下一个优先级配置。若未设置配置文件，会搜索当前目录下是否存在名为tpservice.conf默认配置文件，若设置了配置文件路径，则不会搜索默认配置文件。

2.2.2 SDK 默认配置

以下是TPconfig结构体中的变量的默认配置。

```
version = 1;
account="";
password="";
mountpoint="";
server_host = "pnt.true-point.com";
server_port = 8801U;
server_type = SERVICE_NRTK;
rtcm_interval = 1;
device_id="";
```

```
coordsys = 1;
block_time = 30;
send_gga_with_retry = 1;
retry_interval = 10;
retry_number = 3;
loglevel = LOG_LEVEL_NONE;
authType = TP_CLOUD_CHIP_PRIVATE;
tli_protocol = TLI_TCP;
ssl_config.ssl_option = SSL_VALIDATE_CN | SSL_VALIDATE_CERTS;
ssl_config.ssl_ca_path ="";
support_gga_offset = 1;
```

2.2.3 坐标框架信息

下表中的坐标框架信息仅作说明用途，不代表 SDK 某一版本所固定支持的完整列表。坐标框架由云端动态配置，可能随业务或平台更新而变化；如需确认当前可用框架，请联系真点技术支持。

2.2.3.1 国内NRTK坐标框架信息

坐标框架	coordsys
ITRF2008 参考历元 2016.0	3
WGS84 参考历元 2005.0	12（兼容1）
CGCS2000 参考历元 2000.0	2
ITRF2014 参考历元 2018.0	11
地方CGCS2000	10
ITRF2008 参考历元 2016.0（中移）	6
WGS84 参考历元 2005.0（中移）	4
CGCS2000 参考历元 2000.0（中移）	5

2.2.3.2 海外NRTK坐标框架信息

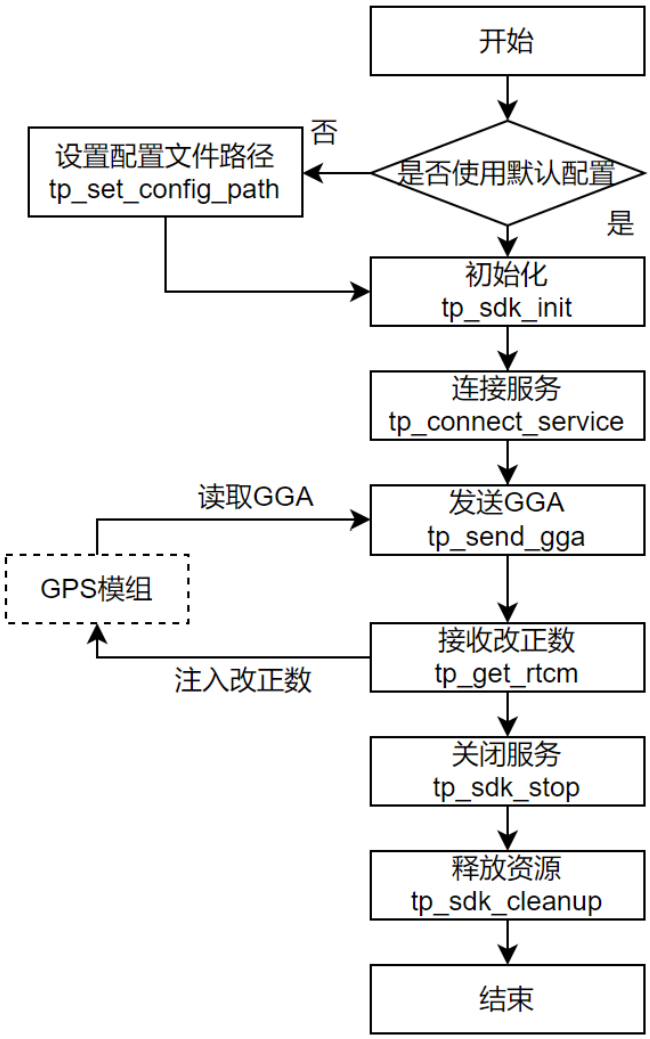
坐标框架	coordsys
ITRF2008 参考历元 2016.0	3
WGS84 当前历元	1
WGS84 参考历元 2005.0	12
ETRS89 参考历元2010.0	100

坐标框架	coordsys
ETRS89 参考历元2010.5	109
ETRS89 参考历元1997.0	106
ETRF93 参考历元1989.0	107

3 接口调用

本章介绍SDK的推荐调用流程及开发须知。

3.1 调用流程



上面流程图中，有以下几点需要注意:

1. `tp_send_gga` 发送 GGA 之后，才能通过 `tp_get_rtcm` 收到对应的 RTCM 数据；连接成功后建议先发送 GGA，再获取 RTCM。

2. `tp_sdk_stop` 之后可调用 `tp_connect_service` 回到连接流程；`tp_sdk_cleanup` 之后需重新调用 `tp_sdk_init`（若设置了配置文件路径，需要在 `tp_sdk_init` 之前重新设置路径）。

3.2 开发须知

1. **配置方式**：SDK 提供 `tp_set_config_path` 设置配置文件路径（支持全路径和相对路径）。若未调用该函数，初始化阶段默认尝试读取当前目录 `tpservice.conf`（若不存在则跳过，使用内置默认参数）。
2. SDK接口`tp_send_gga`和`tp_get_rtcm`可以在不同的线程运行，参考SDK使用示例。
3. 若开启TLS功能，TLS和加密算法需要依赖系统的OpenSSL库。SDK需要的OpenSSL库，目前只适配OpenSSL 3.x版本。
4. 用户首次通过AKAS验证，进行账号和device_id绑定（device_id不能为空）。AKAS可以绑定多个device_id，但绑定数量有一定限额，超出限额后，新的激活请求无法绑定账号，API返回错误信息，如6116。同一套AKAS+DeviceID重复调用该API时，不做新增激活，仅复用之前已经绑定的账号。若需要解除绑定，可联系真点技术支持。
5. 日志回调函数`tp_set_log_cb`需在`tp_sdk_init`及`tp_set_config_path`之前调用，保证日志能够全部打印。
6. `tp_sdk_config`需在`tp_sdk_init`之后，`tp_connect_service`之前。在调用`tp_sdk_config`之前，建议先使用`tp_get_current_config`获取到当前配置，在当前配置上修改。
7. 在重连期间（`tp_send_gga`和`tp_get_rtcm`都可能导致重连），每次重连至多尝试retry_number次，每次重试失败后需要等待retry_interval时间，但这期间是完全阻塞的——调用`tp_get_rtcm`的线程卡住，最大阻塞时间（未包含获取锁时间）约为(retry_number-1) * retry_interval+block_time * retry_number + block_time。
8. `tp_get_current_config`拿到的配置信息，与`tp_sdk_config`设置的可能不同步。如果用户在`tp_connect_service`之后调用`tp_sdk_config`修改配置，内部配置信息是否会更新取决于SDK状态，这个时候可以通过`tp_sdk_config`的返回值判断配置是否更新。
9. sdk使用了OpenSSL库，OpenSSL并没有处理SIGPIPE信号，因此在调用sdk时，需要添加SIGPIPE的处理，例如：`signal(SIGPIPE, SIG_IGN);`。
10. SDK的DNS缓存机制：
    - 在`tp_connect_service`时，如果服务地址有变更、IP列表为空、IP列表中无可用IP，则会触发DNS解析，在DNS解析域名成功后，会缓存最多16条IP（IPv4、IPv6）。
    - 在`tp_connect_service`中会顺序从IP列表中提取IP进行连接，如果失败则将该IP标记为不可用。
    - 在`tp_sdk_cleanup`会清空IP列表。
11. 错误码可能由多种原因导致，如`3040`可能由证书过期导致，若想要查看完整错误信息，可以打开日志。打开日志的步骤：
    - 通过接口`tp_set_log_cb`设置日志的回调函数。

- 设置日志等级log_level（配置文件）/loglevel（TPconfig结构体中变量）。

12. 若block_time设置时间过小，在建立连接时DNS解析可能会超时，从而导致建立连接失败。

4 接口说明

本章介绍的接口主要在tp_sdk_api.h和tp_sdk_status_code.h头文件中声明。本章将详细介绍接口的功能、传入传出参数的意义、调用方法等内容，**其中标有*属于功能增强接口，在SDK提供高精定位服务的基础上，提供更多的功能。**

4.1 设置配置文件路径

4.1.1 接口定义

```
INT32 tp_set_config_path(const INT8* path);
```

4.1.2 接口功能

主要功能是设置配置文件路径，以便读取配置信息。需要在`tp_sdk_init`之前调用。

4.1.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
path	const INT8 *	是	有效的配置文件的路径，且长度不超过255	I

4.1.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.1.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode`转换得到。

错误码	含义
0	成功
1020	参数错误
2356	SDK 已完成初始化后，不允许再修改配置文件路径

错误码	含义
2358	配置文件路径超出允许长度，对于当前接口，路径允许的最大长度是255
2500	获取资源锁失败

4.1.6 使用示例

```
INT32 ret=tp_set_config_path("/etc/conf/tpconfig.ini");
```

4.2 SDK初始化

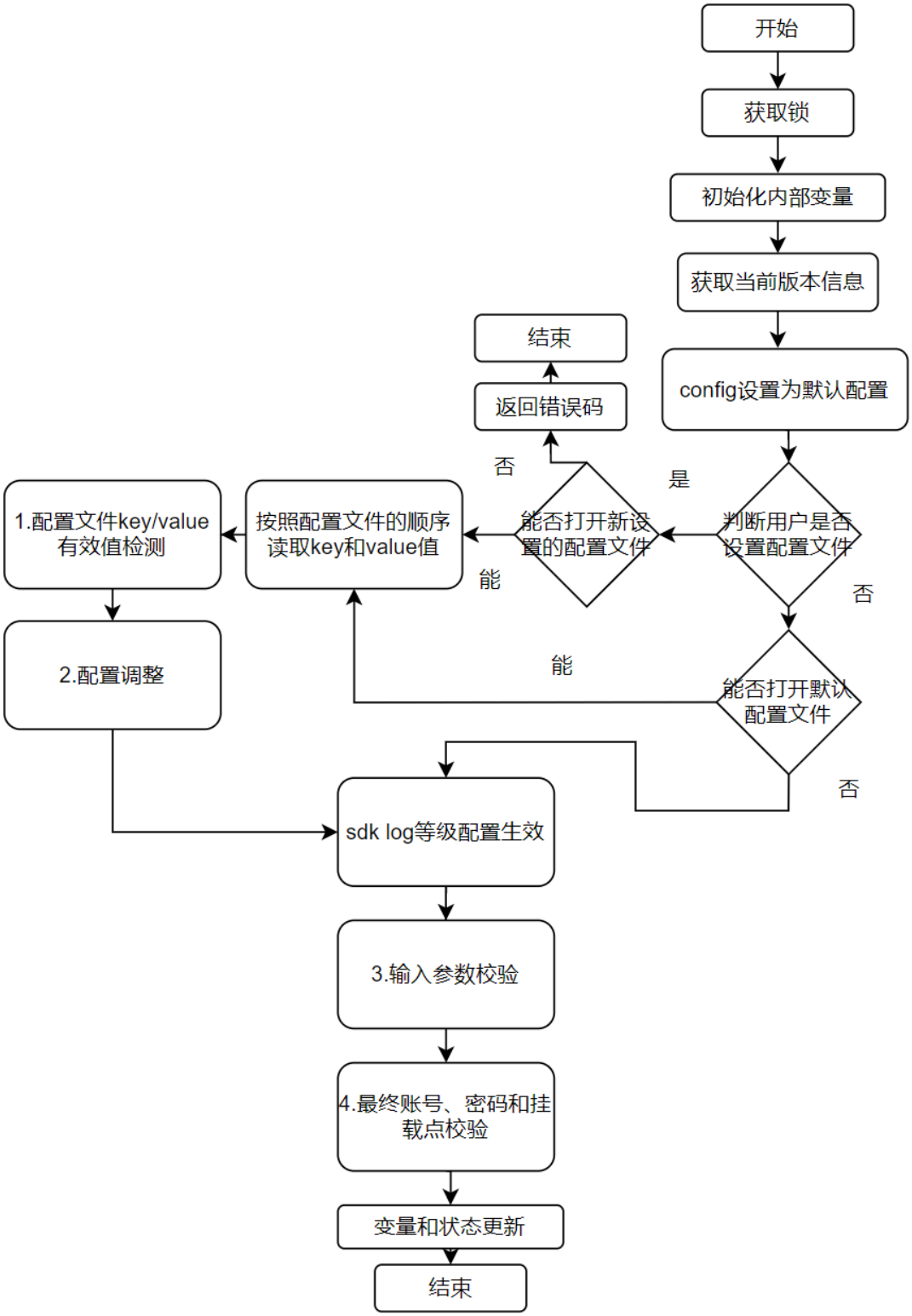
4.2.1 接口定义

```
INT32 tp_sdk_init(const INT8 *user, const INT8 *pwd, const INT8 *device_id, const INT8 *mountpoint, INT32 interval, TP_AUTHTYPE authtype);
```

4.2.2 接口功能

在头文件tp_sdk_api.h中声明，主要功能是初始化SDK参数，通过此接口向SDK注入账号、密码、设备ID等基本信息，输入参数会在后续连接服务过程中引用。INT32等类型的具体定义可查看头文件GH_GSP_Consts.h.

4.2.3 SDK 初始化流程图



4.2.4 检测说明

下文「不为空」均指：指针非 NULL，且字符串长度不为 0。

1. **配置文件 key/value 有效值检测**：按配置文件中 key 与 value 的出现顺序逐项校验；任一项不通过则立即返回错误，不再校验后续配置项。

   - **server**：不为空；若为 `host:port` 格式，则 host 不为空，且 port 为有效的 32 位正整数；若为单独 host，则 host 不为空。
   - **log_level**：须为 `verbose`、`debug`、`info`、`warning`、`error`、`none` 之一。
   - **transport_protocol**：须为 `tcp` 或 `ssl`。
   - **account、password、mount_point、device_id、server_ssl_path**：均不为空。
   - **数值型配置项**：`rtcm_interval`、`coordinate_sys`、`block_time`、`send_gga_with_retry`、`retry_interval`、`retry_number`、`auth_type`、`support_gga_offset` 须为有效正整数或0，并满足：
     - `rtcm_interval` 取值范围 0～254。
     - `coordinate_sys` 大于 0。
     - `block_time` 大于 0。
     - `send_gga_with_retry` 为 0 或 1。
     - `auth_type` 为 `TP_CLOUD_CHIP_PRIVATE` 或 `TP_CLOUD_CHIP_AKAS`。
     - `transport_protocol` 为枚举 `TP_TLI_PROTOCOL` 的有效值。
     - `support_gga_offset` 为 0 或 1。
   - **server_type**：须为 `SERVICE_NRTK`。
   - **server_ssl_verify**：目前支持 `cn`、`cert` 和 `none`，`cn`和`cert`可选一个或同时选择，同时选择以逗号隔开；若出现 `none`，则不能出现`cn`和`cert`。

2. **配置调整**：校验合并后的 config：

   - **rtcm_interval**：若 `rtcm_interval >= 0` 且 `block_time < rtcm_interval + 2`，则将 `block_time` 调整为 `block_time + rtcm_interval + 2`。
   - **port**：若传输协议为 SSL 且未设置端口，则默认端口为 18801。若设置传输协议是 SSL 且端口设置为 8801，则返回错误码。
   - **auth_type**：若设置 `auth_type` 为 `TP_CLOUD_CHIP_AKAS`，但是未设置传输协议为 SSL，则返回错误码。

3. **输入参数校验**：根据用户传入参数覆盖配置，校验顺序为：`interval` → `mountpoint` → `user` → `pwd` → `device_id` → `authtype`。任一项失败则不再继续，直接返回错误。

   - **interval**：为 -1 时使用已有配置并继续下一项；小于 0 或大于 254 则返回错误；否则写入传入值。

- **mountpoint**：为 NULL 时使用已有配置；长度为 0 或大于 63 则返回错误。
- **user**：为 NULL 时使用已有配置；长度为 0 或大于 63 则返回错误。
- **pwd**：为 NULL 时使用已有配置；长度为 0 或大于 63 则返回错误。
- **device_id**：为 NULL 时使用已有配置；大于 63 则返回错误。
- **authtype**：为 -1 时使用已有配置；若不为 `TP_CLOUD_CHIP_PRIVATE` 或 `TP_CLOUD_CHIP_AKAS` 则返回错误。

4. **最终账号、密码与挂载点校验**：在合并后的 config 中依次校验账号、密码、挂载点均不为空；任一项不通过则直接返回错误码。

4.2.5 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
user	const INT8 *	否	服务账号，传入 NULL 时使用配置文件中的用户名	I
pwd	const INT8 *	否	服务密码，传入NULL时使用配置文件中密码	I
device_id	const INT8 *	否	设备 ID，传入 NULL 时使用配置文件中设置。	I
mountpoint	const INT8 *	否	挂载点，传入NULL时使用配置文件中设置	I
interval	INT32	否	改正数播发时间间隔（s）， 标准版本有效范围[0,254]。 若使用省流版本，仅支持[1,5]。 设置-1时，使用配置文件或默认配置。 若设置为0，则为一发一收模式， 即只在发送GGA后才会获取改正数。 推荐interval的设置： 1.当选择挂载点为RTCM开头时， 播发频率由SDK参数interval设置， 推荐设置为10以内， 否则可能影响终端定位效果。 2.当您选择使用云芯一体模式降低流量消耗， 此时挂载点推荐使用TPRTCM3， 改正数的播发频率为3s。 此模式将有效降低流量消耗70%以上， 节约用户成本。	I
authtype	TP_AUTHTYPE	否	参考tp_sdk_api.h对应定义，当前版本仅支持 TP_CLOUD_CHIP_PRIVATE（私有协议）和 TP_CLOUD_CHIP_AKAS(Access	I

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
			key和Access secret)。 若使用配置文件或者默认配置， 可以设置为-1。	

4.2.6 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.2.7 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
1021	需要打开TLS配置
2001	SDK已经初始化
2300	账号为空
2302	密码为空
2304	挂载点为空
2306	服务地址为空或端口无效
2309	服务端口与传输协议不匹配（如 TLS 须配合约定端口）
2310	加载配置文件失败
2312	无效配置
2313	无效配置： 支持GGA偏移计算
2314	无效配置： 日志等级
2316	无效配置： 服务地址
2318	无效配置： 传输协议
2320	无效配置：SSL验证项

错误码	含义
2322	无效配置： SSL证书路径
2324	无效配置：账号
2326	无效配置：密码
2328	无效配置： 挂载点
2330	无效配置：设备ID
2336	无效配置：差分数据播发间隔
2338	无效配置： 坐标系
2340	无效配置： 网络阻塞等待时间
2342	无效配置： 服务类型
2344	无效配置： 自动重连
2346	无效配置： 重连间隔
2348	无效配置： 认证方式
2352	无效配置： 重连次数
2357	配置路径不存在
2500	获取资源锁失败
4000	服务类型无效
4120	坐标无效
4125	参数过长

4.2.8 使用示例

```
INT32 ret=tp_sdk_init("user","pwd","devID","mountpoint",1,TP_CLOUD_CHIP_PRIVATE);
```

4.3 获取SDK当前配置信息

4.3.1 接口定义

```
INT32 tp_get_current_config(TPconfig *config);
```

4.3.2 接口功能

在头文件tp_sdk_api.h中声明，通过该接口能够获取当前sdk的配置信息，tp_sdk_init后使用。**ssl_config.ssl_ca_path默认配置为空，可以使用系统默认的证书链。当ssl_config.ssl_ca_path为空时，tp_get_current_config返回结构体中会使用全0xFF填充对应的数组。**

4.3.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
config	TPconfig *	是	调用方提供的 TPconfig 缓冲区，不可为 NULL	O

4.3.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.3.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口tp_get_errorcode** 转换得到。

错误码	含义
0	成功
1020	参数错误
2000	SDK 尚未初始化
2500	获取资源锁失败

4.3.6 使用示例

```
TPconfig config_read;
ret = tp_get_current_config(&config_read);
```

4.4 功能配置

4.4.1 接口定义

```
INT32 tp_sdk_config(const TPconfig *config)
```

4.4.2 接口功能

在头文件 tp_sdk_api.h 中声明。用于在连接前写入通用配置，需要在 `tp_sdk_init` 之后调用；**字段含义与 TPconfig、配置文件一致**，见下表及 `tp_sdk_api.h`。
对比之前版本，TPconfig数据结构添加了新的变量。为了兼容之前的版本、防止未定义数据，**需要先将数据设置成-1**，此时不会改动已有配置（使用之前已经选择的默认配置或配置文件中配置），再配置 block_time 和 coordsys。

4.4.3 参数说明

TPconfig结构体成员名称	参数类型	是否必填	描述	输入/输出(I/O)
version	UINT32	否	版本号，用于向前兼容，目前为保留字段	I
account	INT8数组	否	账号信息，详见SDK初始化参数表usr变量	I
password	INT8数组	否	密码信息，详见SDK初始化参数表pwd变量	I
mountpoint	INT8数组	否	挂载点信息，详见SDK初始化参数表mountpoint变量	I
server_host	INT8数组	否	服务端信息	I
server_port	UINT32	否	服务端端口号，port需大于0并小于等于65535。	I
server_type	UINT32	否	服务类型，参考tp_sdk_api.h中TP_SERVICE_TYPE的枚举范围，目前仅支持SERVICE_NRTK。	I
rtcm_interval	INT32	否	服务端数据下发时间间隔，详见SDK初始化参数表interval变量	I
device_id	INT8数组	否	设备ID，详见	I

TPconfig结构体成员名称	参数类型	是否必填	描述	输入/输出(I/O)
			SDK初始化参数表device_id变量	
coordsys	INT32	否	坐标框架，详见坐标框架信息。 坐标系实际运行情况 请联系技术支持。	I
block_time	INT32	否	网络传输数据最大等待时间， 单位是秒。 SDK基础参数配置， 注意配置的阻塞时间必须大于 RTCM播发时间间隔， 若blocktime设置时间少于 或接近播发间隔 可能影响rtcm接收。 若有其他配置和问题 可联系真点技术支持。	I
send_gga_with_retry	INT32	否	是否在发送GGA失败时进行重连， 0不进行重连，1 进行重连。	I
retry_interval	INT32	否	重试间隔时间，单位秒。	I
retry_number	INT32	否	连接重试次数	I
loglevel	UINT32	否	日志配置， 参考tp_sdk_api.h中定义 TP_LOG_LEVEL枚举范围。	I
authType	INT32	否	认证类型	I
tli_protocol	UINT32	否	Transport Layer Interface配置， 参考tp_sdk_api.h 中 TP_TLI_PROTOCOL的枚举范围， 当前版本仅支持tcp和ssl。	I
ssl_config	St_ssl_config_t	否	TLS配置， 具体定义参考tp_sdk_api.h， ssl_ca_path中路径长度 不超过255。	I
support_gga_offset	INT32	否	是否启用 GGA 位置随机偏移： 0 关闭，1 开启	I

4.4.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.4.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
1020	参数错误
1021	需要打开TLS配置
2000	SDK 尚未初始化
2300	账号为空
2302	密码为空
2304	挂载点为空
2306	服务地址为空或端口无效
2309	服务端口与传输协议不匹配（如 TLS 须配合约定端口）
2313	无效配置：支持GGA偏移计算
2314	无效配置：日志等级
2318	无效配置：传输协议
2320	无效配置：SSL验证项
2322	无效配置：SSL证书路径
2336	无效配置：差分数据播发间隔
2340	无效配置：网络阻塞等待时间
2344	无效配置：自动重连
2346	无效配置：重连间隔
2348	无效配置：认证方式
2352	无效配置：重试次数

错误码	含义
2354	服务已连接，配置无效
2500	获取资源锁失败
4000	服务类型无效
4120	坐标无效
4135	服务已终止

4.4.6 使用示例

```
// 二选一
//1. 在当前配置上改动
TPconfig current_config={0};
ret = tp_get_current_config(&current_config);
if (ret != 0)
{
  printf("invoke tp_get_current_config return %d, errcode %d: %s\n", ret,
          tp_get_errorcode(ret), tp_get_errormessage(ret));
  tp_sdk_cleanup();
  return 1;
}
//可以添加打印查看具体内容
//按照需要给current_config赋值
ret = tp_sdk_config(&current_config);

//2.全部设置成-1,即使用默认设置
TPconfig config;
memset(&config,-1,sizeof(config));
//按照需要给config赋值
ret = tp_sdk_config(&config);
```

4.5 连接服务

4.5.1 接口定义

```
INT32 tp_connect_service(const INT8 *addr, const INT32 port)
```

4.5.2 接口功能

在头文件tp_sdk_api.h中声明，此接口是为了连接真点高精度服务，需要配置服务域名和端口号。

4.5.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
addr	const INT8 *	否	服务域名或 IP 地址，若传入 NULL，使用默认配置或者配置文件中的值	I
port	const INT32	否	端口信息，取值(0,65535]。小于等于0时使用默认值或者配置文件中的值。	I

4.5.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.5.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
1020	参数错误
2000	SDK 尚未初始化
2022	Protobuf 处理数据错误：解析服务响应
2306	服务地址为空或端口无效
2308	阻塞时间小于播发间隔
2309	服务端口与传输协议不匹配（如 TLS 须配合约定端口）
2500	获取资源锁失败
3031	连接已建立
3040	超时（DNS解析超时、服务器连接超时等）
3080	DNS 解析错误
4120	坐标无效
4125	参数过长

错误码	含义
4255	IPC发送接收错误
6000～6999	平台业务错误（见错误码）

4.5.6 使用示例

```
INT32 ret = tp_connect_service("xxx.xxx.xx.xxx",8801);
```

4.6 发送GGA

4.6.1 接口定义

```
INT32 tp_send_gga(const INT8 *buf, INT32 len);
```

4.6.2 接口功能

在头文件tp_sdk_api.h中声明，可通过此接口向服务器发送GGA数据。
`tp_send_gga`具有以下功能：

1. 能够进行 GGA 的简单校验，只有输入的 GGA 符合 NMEA0183 格式才会发送。
2. 对 GGA 进行随机偏移，保护个人信息。
3. 能够进行网络重连：
    - `tp_send_gga` 使用用户最近发送的有效 GGA（通过上述功能 1 验证）进行重连，若用户没有有效 GGA，则返回错误。
    - **send_gga_with_retry=0**：不进行重连，此时 **retry_number** 不参与语义。
    - **send_gga_with_retry=1**：允许失败后自动重连，retry_number 为一次重连尝试的次数，retry_interval 为一次 retry_number 的时间间隔（不包括首次）。
    - 若返回网络错误或平台错误码6046，SDK将重新尝试连接。
    - 当返回其他平台错误（如账号密码错误）时，则不再进行重连。

4.6.3 tp_send_gga流程

1. 获取锁。若失败，直接返回锁相关错误码。
2. 验证输入的 GGA 是否有效。若无效，选择最近发送的有效 GGA，若无法获取，返回失败。

3. 检查 SDK 状态是否正常。
4. 根据 `support_gga_offset` 处理上报内容：
    - 开启偏移：对待发送的 GGA 添加偏移，生成脱敏 GGA。
    - 关闭偏移：直接发送原始 GGA，并保存最近一次有效 GGA。
5. 发送 GGA：
    - 若发送成功则释放锁并返回 0。
    - 若发送失败：
        - 释放 GGA 发送锁，获取重连锁，若有其他线程在重连，则获取锁失败，返回错误码。
        - 判断发送 GGA 失败的错误类型：
            - 若是网络错误或不在服务范围内（6046），尝试重连。
            - 若是账号密码错误等云端错误码（6046除外），则释放锁并直接返回对应错误码。
        - 若重连，按重连参数（`retry_interval` 和 `retry_number`）进行重连:
            - 重连失败或超出最大次数返回错误码。
            - 重连成功后，重新发送GGA。若后续发送GGA仍失败，则直接返回错误码。
        - 释放锁。

4.6.4 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
buf	const INT8 *	是	GGA数据，符合NMEA0183格式	I
len	INT32	是	GGA数据长度，目前最大允许长度为199	I

4.6.5 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.6.6 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode`转换得到。

错误码	含义
0	成功
1010	执行错误
2000	SDK 尚未初始化
2500	获取资源锁失败
3010	网络错误
4020	无效 GGA
4135	服务已终止
6000～6999	平台业务错误（见错误码）

4.6.7 使用示例

```
INT32 ret= tp_send_gga(gga, strlen(gga));
```

4.7 获取RTCM改正数

4.7.1 接口定义

```
INT32 tp_get_rtcm(INT8 *buf, INT32 *len)
```

4.7.2 接口功能

在头文件tp_sdk_api.h中声明，主要功能是从服务器获取RTCM改正数。同`tp_send_gga`一样，`tp_get_rtcm`支持重连操作。若接口调用返回0，获取的`len`也是0，一般是在接口调用的过程中发生了重连，此时需要重新发送GGA才能收到RTCM数据。

4.7.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
buf	INT8 *	是	用于存放接收到的RTCM数据的缓存	O

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
len	INT32 *	是	输入时为接收RTCM缓存的长度， 输出时为实际接收到的RTCM长度。	I/O

4.7.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.7.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
1020	参数错误
2000	SDK 尚未初始化
2500	获取资源锁失败
3010	网络错误
3040	超时
4020	无效 GGA
4031	未上传GGA导致改正数接收超时
4135	服务已终止
6000～6999	平台业务错误（见错误码）

4.7.6 使用示例

```
char rtcm_buf[2048];
rtcm_len = sizeof(rtcm_buf);
memset(rtcm_buf, 0, rtcm_len);
ret= tp_get_rtcm(rtcm_buf, &rtcm_len);
```

4.8 查询功能*

4.8.1 接口定义

```
INT32 tp_sdk_query(TP_QUERY_ID id, St_sdk_response_t *stResponse, St_sdk_reserved_t *reserved)
```

4.8.2 接口功能

主要功能是查询账号信息、算法引擎授权状态、SDK版本号等信息。此接口的类型中，服务状态信息是对账号信息类型的升级。服务状态信息类型可同时查询到账号购买的多种类型服务信息，而账号信息类型的结果只有一个服务类型，如果账号购买了多种类型的服务，会随机返回其中一种服务的状态信息。
注：当前版本仅支持 查询SDK版本

4.8.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
id	TP_QUERY_ID	是	查询功能点类型编码：GET_SDK_VERSION(SDK版本查询)	I
stResponse	St_sdk_response_t*	是	返回查询结果	O
reserved	St_sdk_reserved_t*	否	预留位，可使用NULL占位	-

4.8.4 返回值说明

返回值类型	返回值说明
INT32	失败返回retcode，成功返回0

4.8.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode`转换得到。

错误码	含义
0	成功
1020	参数错误
2500	获取资源锁失败

4.8.6 使用示例

```
St_sdk_response_t stResponse = {0};
memset(&stResponse, 0, sizeof(stResponse));
ret = tp_sdk_query(GET_SDK_VERSION, &stResponse, NULL);
if (tp_get_errorcode(ret) == TP_SDK_FUNC_INVOKE_OK)
{
      printf("True-Point SDK Version: %s\n", stResponse.uResult.com_result.buf);
}
```

4.9 关闭服务

4.9.1 接口定义

```
INT32 tp_sdk_stop()
```

4.9.2 接口功能

在头文件 tp_sdk_api.h 中声明，主要功能是关闭服务，断开连接。

4.9.3 参数说明

无需传递参数

4.9.4 返回值说明

返回值类型	返回值说明
INT32	成功返回0，失败返回retcode

4.9.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
2500	获取资源锁失败

4.9.6 使用示例

```
ret = tp_sdk_stop();
```

4.10 资源释放

4.10.1 接口定义

```
int tp_sdk_cleanup()
```

4.10.2 接口功能

清理SDK所占用资源。

4.10.3 参数说明

无需传递参数

4.10.4 返回值说明

返回值类型	返回值说明
int	失败返回retcode，成功返回0

4.10.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口**`tp_get_errorcode` 转换得到。

错误码	含义
0	成功
2500	获取资源锁失败

4.10.6 使用示例

```
ret = tp_sdk_cleanup();
```

4.11 注册socket配置回调，用于对socket进行配置

4.11.1 接口定义

```
void tp_set_socket_cb(void(*pcb_fun)(int fd))
```

4.11.2 接口功能

在头文件tp_sdk_api.h中声明，主要功能是设置文件描述符回调函数，用于对网络部分做自定义操作。
建议调用该接口在tp_sdk_init之前调用。

4.11.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
pcb_fun	void(*)(int)	是	用于在网络连接前， 传回socket文件描述符， 方便客户配置socket属性。	I

4.11.4 返回值说明

无

4.11.5 接口常见错误码

无

4.11.6 使用示例

```
tp_set_socket_cb(printsockfd);
```

4.12 获取网络文件描述符

4.12.1 接口定义

```
INT32 tp_get_network_fd(INT32 fdType)
```

4.12.2 接口功能

获取网络文件描述符

4.12.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
fdType	INT32	是	保留	-

4.12.4 返回值说明

返回值类型	返回值说明
INT32	成功时返回 **≥0** 的套接字描述符，失败时返回 **<0** 的 retcode， 请用 **tp_get_errorcode(ret)** 解析为错误码（-1除外，-1为无效的套接字）；若解析结果与现象不符，请结合 **tp_get_errormessage(ret)** 或联系技术支持

4.12.5 接口常见错误码

下面是接口常见的错误码，这些错误码由接口的返回值retcode经**返回值转错误码接口tp_get_errorcode** 转换得到。

错误码	含义
0	成功
2000	SDK 尚未初始化
2500	获取资源锁失败
4005	服务尚未准备好（未完成登录、激活等前置步骤）
4135	服务已终止

4.12.6 使用示例

```
printf("network fd is %d\n",tp_get_network_fd(0));
```

4.13 返回值转错误码*

4.13.1 接口定义

```
int tp_get_errorcode(int retcode);
```

4.13.2 接口功能

在头文件tp_sdk_status_code.h中声明，主要功能是将接口返回的retcode转换成错误码表对应的错误码。

4.13.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
retcode	int	是	接口返回的retcode	I

4.13.4 返回值说明

返回值类型	返回值说明
int	返回错误码

4.13.5 接口常见错误码

将retcode转化为错误码，无针对该函数的错误码。

4.13.6 使用示例

```
int er = tp_get_errorcode(ret);
```

4.14 获取返回值对应的错误信息*

4.14.1 接口定义

```
const char *tp_get_errormessage(int retcode)
```

4.14.2 接口功能

在头文件tp_sdk_status_code.h中声明，主要功能是将接口返回的retcode转换成可理解的错误描述信息。

4.14.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
retcode	int	是	接口返回的retcode	I

4.14.4 返回值说明

返回值类型	返回值说明
const char*	返回返回值对应的描述信息

4.14.5 接口常见错误码

无

4.14.6 使用示例

```
ret = tp_connect_service(NULL, 0);
if (ret != 0)
{
    fprintf(stderr, "tp_connect_service failed: ret=%d  errcode=%d  msg=\"%s\"\n",
          ret, tp_get_errorcode(ret),
          tp_get_errormessage(ret) ? tp_get_errormessage(ret) : "(null)");
    tp_sdk_cleanup();
    return 1;
}
printf("[SDK] connected\n");
```

4.15 注册日志输出回调函数*

4.15.1 接口定义

```
void tp_set_log_cb(void (*callback)(UINT64 timestamp, INT32 level, const char *tag, const char *mes
sage));
```

4.15.2 接口功能

在头文件tp_sdk_api.h中声明，用于注册日志输出回调；传入 NULL 则不进行输出。回调在日志过滤和格式化后调用，参数为：时间戳、级别、标签、完整格式化后的单行消息缓冲区（只读使用，勿长期保存指针）。

4.15.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
callback	void (*)(UINT64 timestamp, INT32 level, const char *tag, const char *message)	否	timestamp：时间戳信息 level：log等级 tag：标签 message：完整格式化后的单行消息缓冲区	I

4.15.4 返回值说明

无

4.15.5 接口常见错误码

无

4.15.6 使用示例

```
...
static void dummy_log_register_cb(UINT64 timestamp, INT32 level, const char *tag, const char *mes
sage)
{
   //此处添加处理
}
...
//注册回调
 tp_set_log_cb(dummy_log_register_cb);
```

4.16 注册日志格式化回调*

4.16.1 接口定义

```
void tp_set_log_formatter(const void *reserved_null_only);
```

4.16.2 接口功能

在头文件 tp_sdk_api.h 中声明，目前作为一个保留接口。

4.16.3 参数说明

参数名称	参数类型	是否必填	描述	输入/输出(I/O)
reserved_null_only	NULL	是	保留，目前只支持NULL	I

4.16.4 返回值说明

无

4.16.5 接口常见错误码

无

4.16.6 使用示例

```
tp_set_log_formatter(NULL);
```

5 错误码

常见错误码表如下，与 `tp_sdk_status_code.h` 中 `St_tp_status_code_t` 枚举数值及注释一致；推荐集成时直接使用头文件中的枚举常量。表中 **6000～6999** 为平台业务返回码区间：头文件仅列出 SDK 常用的若干映射（如 6046、6100 等），其余平台子码由服务端下发，完整清单请联系真点技术支持获取。

错误码	描述	出现原因及操作
0	执行成功	
1-999	系统错误	如Linux系统，返回errno对应值等
1010	执行错误	
1020	参数错误	
1021	需要打开TLS配置	
1999	SDK未知错误	
2000	SDK 尚未初始化	
2001	SDK 已经初始化	
2010	protobuf传入内存不足	
2020	Protobuf处理数据错误	
2022	Protobuf处理数据错误：解析服务响应	
2024	Protobuf处理数据错误：封装开启NRTK服务请求	
2026	Protobuf处理数据错误：封装上报GGA	
2028	Protobuf处理数据错误：解析差分数据	
2030	Protobuf处理数据错误：封装登录请求	
2032	Protobuf处理数据错误：获取随机数	
2100	MSM4传入内存不足	
2200	SHA256错误	
2300	账号为空	
2302	密码为空	
2304	挂载点为空	
2306	服务地址为空或端口无效	

错误码	描述	出现原因及操作
2308	阻塞时间小于播发间隔	
2309	服务端口与传输协议不匹配（如 TLS 须配合约定端口）	
2310	加载配置文件失败	
2312	无效配置	
2313	无效配置：支持GGA偏移计算	
2314	无效配置：日志等级	
2316	无效配置：服务地址	
2317	无效配置：MSM2支持	
2318	无效配置：传输协议	
2320	无效配置：SSL验证项	
2322	无效配置：SSL证书路径	
2324	无效配置：账号	
2326	无效配置：密码	
2328	无效配置：挂载点	
2330	无效配置：设备ID	
2332	无效配置：GPS设备路径	
2334	无效配置：GPS设备波特率	
2336	无效配置：差分数据播发间隔	
2338	无效配置：坐标系	
2340	无效配置：网络阻塞等待时间	
2342	无效配置：服务类型	
2344	无效配置：自动重连	
2346	无效配置：重连间隔	
2348	无效配置：认证方式	
2350	无效配置：GGA上报间隔	
2352	无效配置：重试次数	
2354	服务已连接，配置无效	

错误码	描述	出现原因及操作
2356	SDK已完成初始化后，不允许再修改配置文件路径	
2357	配置路径不存在	
2358	配置路径过长	
2500	获取资源锁失败	
2503	重连次数超过最大重连次数	
3010	网络错误	
3020	连接错误	
3030	TCP链接尚在建立中	
3031	连接已建立	
3040	超时	
3050	不支持传输协议	
3060	连接被中断	
3070	需要重新尝试接收或发送数据	
3080	DNS解析错误	
3400	SSL初始化错误	
3410	SSL握手进行中	
3420	SSL握手错误	
3430	SSL加载证书错误	
3440	SSL上下文无效	
3450	SSL内部错误	
3460	SSL底层系统调用错误	
3999	OSAL未知错误	
4000	服务类型无效	
4005	服务尚未准备好（未完成登录、激活等前置步骤）	
4010	发送认证信息错误	
4011	private 服务未初始化	
4012	服务已初始化，禁止重复 init	

错误码	描述	出现原因及操作
4013	已登录，禁止重复登录	
4014	已激活连接，禁止重复 connect	
4015	未激活，禁止 send/recv	
4020	无效GGA	
4030	发送GGA错误	
4031	未上传GGA导致改正数接收超时	
4040	封装错误	
4100	不支持的播发间隔	
4110	阻塞时间小于播发间隔	
4120	坐标无效	
4125	参数过长	
4130	状态无效	
4135	服务已终止	
4140	无效的服务地址	
4145	无效的配置	
4150	无效的帧	
4200	epoll错误	
4205	epoll添加事件错误	
4210	epoll删除事件错误	
4215	epoll等待事件错误	
4220	epoll创建错误	
4245	IPC写入错误	
4250	IPC错误	
4255	IPC发送接收错误	
4260	IPC轮询错误	
4265	IPC读取错误	
4270	IPC创建错误	

错误码	描述	出现原因及操作
4300	DNS错误	
4305	DNS发送接收错误	
4310	DNS线程启动错误	
4350	未处理的命令码	
4355	工作线程启动错误	
4360	未处理的命令码	
4365	无效的帧	
4999	未知服务错误	
6000	平台返回码的最小值	
6002	连接服务失败	
6010	网络连接超时	
6046	服务未覆盖	
6100	账号密码错误	账号不存在、账号存在密码错误、AK不存在、AK存在AS错误等
6105	账号类型错误	
6106	该账号服务已用尽，请及时续费	
6107	该账号被锁定，暂无法使用服务	
6111	该账号无法自动激活，请手动激活后再访问服务	
6113	该账号并发数已达上限	
6114	账号已经登录，重复登录	
6115	登录过于频繁	
6116	没有可分配的账号	
6120	登录参数不能为空	账号、密码和挂载点均不能为空
6304	坐标框架错误	
6305	接入点不存在	
6306	接入点不可用	
6307	接入点错误	
6308	账号未购买所使用服务	

错误码	描述	出现原因及操作
6309	暂不支持该服务	
6310	服务已开启	
6311	服务未开启	
6330	账号已经禁用	
6399	参数错误	
6502	请先登录服务账号	
6510	请求已过期，请重新连接	
6601	当前没有可升级固件版本	
6602	未查询到该固件版本信息	
6999	平台返回码的最大值	
9999	未知错误	

6 常见问题

Q1：差分数据上传的频率调用是否有限制?

A1：在SDK初始化的参数interval可以设置上传频率。用户可以根据实际差分数据的播发频率进行设置，建议上传频率为1s（小于阻塞时间block_time）。

Q2：如何开启TLS?

A2：包括以下步骤

（1） 在可执行程序目录下创建一个配置文件：tpservice.conf(sdk默认路径)或使用`tp_set_config_path`配置文件路径；

（2）在配置文件中增加下面几项配置：

```
transport_protocol = ssl
server_ssl_verify = cert,cn
```

（3） 若配置文件中设置了端口，可直接在配置文件中将端口改为18801；若配置文件未设置，可以在连接时候通过接口`tp_connect_service`调用设置端口18801;若未设置端口，打开了ssl，默认也会用到18801。

Q3：如何修改配置参数?

A3：推荐方式是通过`tp_get_current_config`先获取当前配置，这时可根据需要确定修改哪些配置，再通过接口`tp_sdk_config`设置配置。

```
TPconfig current_config={0};
 // 获取目前的配置
ret = tp_get_current_config(&current_config);
if (ret != 0)
{
  printf("invoke tp_get_current_config return %d, errcode %d: %s\n", ret,
            tp_get_errorcode(ret), tp_get_errormessage(ret));
  tp_sdk_cleanup();
  return 1;
}
//可按照需要添加打印查看current_config的函数
//按照需要给current_config，如下面更新了block_time
current_config.block_time = 3;
//调用sdk端口更新设置
ret = tp_sdk_config(&current_config);
```

Q4：什么情况下会出现3040报错?

A4：3040是对应网络数据超时的描述，一般是在超过规定的超时等待时间后，仍然没有收到数据，就会触发这类报错。若检查当前网络没有问题，可保存日志，联系真点技术支持查看。

Q5:如何绑定网卡?

A5:可以在`tp_set_socket_cb`的回调函数中调用setsockopt绑定设备（SO_BINDTODEVICE），示例如下：

```
/*
  * socket 回调函数，会在socket创建后立即被调用
  */
  void NtripImplSdk::socketBind(int fd)
  {
    // 绑定网络设备
    if (!interface_name.empty())
    {
      struct ifreq ifr;
      memset(&ifr, 0, sizeof(ifr));
      strncpy(ifr.ifr_name, interface_name.c_str(), IFNAMSIZ - 1);

      if (setsockopt(fd, SOL_SOCKET, SO_BINDTODEVICE, (void *)&ifr, sizeof(ifr)) != 0)
      {
        printf("setsockopt(SO_BINDTODEVICE) failed: %s\n", strerror(errno));
        // 注意：可能需要 root 权限或 CAP_NET_RAW capability
      }
      else{
        printf("setsockopt(SO_BINDTODEVICE) successed: %s\n", interface_name.c_str());
      }
    }

    // 绑定网络端口
    if (localPort != 0)
    {
      struct sockaddr_in address; // 用于存储地址信息的结构体
      int addrlen = sizeof(address);  // address 结构体的长度
      int opt = 1;

      // 设置地址复用
      if (setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) != 0)
      {
        printf("setsockopt(SO_REUSEADDR) failed: %s\n", strerror(errno));
      }
      else
      {
        printf("setsockopt(SO_REUSEADDR) successed\n");
      }

      // 设置端口复用
#ifdef SO_REUSEPORT
      if (setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt)) != 0)
      {
```

```
            printf("setsockopt(SO_REUSEPORT) failed: %s\n", strerror(errno));
        }
        else
        {
            printf("setsockopt(SO_REUSEPORT) successed\n");
        }
#endif

        // 设置 SO_LINGER，慎用！！！
        struct linger ling;
        ling.l_onoff = 1;  // 启用 SO_LINGER
        ling.l_linger = 0; // 设置延迟时间

        if (setsockopt(fd, SOL_SOCKET, SO_LINGER, &ling, sizeof(ling)) != 0)
        {
            printf("setsockopt(SO_LINGER) failed: %s\n", strerror(errno));
            // 即使设置失败，也继续关闭
        }
        else
        {
            printf("setsockopt(SO_LINGER) successed, onoff %d, linger %d\n", ling.l_onoff, ling.l_linger);
        }

        // 绑定端口
        address.sin_family = AF_INET;       // IPv4 地址族
        address.sin_addr.s_addr = INADDR_ANY; // 绑定到所有可用的本地网络接口
                                    // 也可以使用 inet_addr("127.0.0.1") 绑定到特定 IP
        address.sin_port = htons(localPort);  // 将端口号从主机字节序转换为网络字节序
        while (1)
        {
            if (bind(fd, (struct sockaddr *)&address, addrlen) != 0)
            {
                printf("socket bind %d to port %d failed: %s\n", fd, localPort, strerror(errno)); // 如果端口被
占用，这里会失败并打印 "Address already in use"
                sleep(1);
                continue;
            }
            else
            {
                printf("socket bind %d to port %d successfully\n", fd, localPort);
                break;
            }
        }
    }
}
```

7 SDK使用示例

示例如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>

#include "tp_sdk_api.h"
#include "osal_time.h"
#include "tp_sdk_status_code.h"

// 控制接收rtcm进程是否结束
volatile int isstop = 0;

// 演示用的模拟gga数据
static const char s_default_mock_gga[] =
    "$GNGGA,090230.00,4004.7061,N,11614.0472,E,4,51,0.4,66.4860,M,-8.4923,M,1.0,387*4D\r\n";

// 使用真实的写入rtcm函数替换
INT32 ttywrite(const UINT8 *buf, INT32 buf_len) {
    return buf_len;
}

// 使用真实的初始化串口函数替代
int init_tty(const UINT8 *serial, int baudrate) {
    return 1;
}

// 按需实现
void log_callback(UINT64, INT32, const char *, const char *message) {
    printf("%s", message);
}

// 使用真实的获取gga方法替换
INT32 ttyread(UINT8 *buf, INT32 buf_len) {
    INT32 len = strlen(s_default_mock_gga);
    memcpy(buf, s_default_mock_gga, (size_t)len);
    return len;
}

// 按需实现
void set_socket_callback(int fd) {
```

```
    printf("fd is %d\n", fd);
}

// rtcm接收任务
void handle_rtcm_recv() {
    char rtcmBuf[1024 * 10];
    int rtcm_len = 0;
    while (isstop == 0) {
        /* receive RTK correction data from True-Point server. */
        rtcm_len = sizeof(rtcmBuf) - 1;
        rtcmBuf[0] = 0;
        int rtcmRet = tp_get_rtcm((INT8*)rtcmBuf, &rtcm_len);
        printf("invode tp_get_rtcm return %d, rtcm_len %d\n", tp_get_errorcode(rtcmRet), rtcm_len);
        if (rtcmRet == 0) {
            /* transmit RTK correction data to GPS module.*/
            if (rtcm_len > 0) {
                /* 收到rtcm消息，写入rtcm */
                if (ttywrite((const UINT8 *)rtcmBuf, rtcm_len) < 0) {
                    printf("write serial failed\n");
                }
            } else {
                printf("no rtcm data received !\n");
                // 防止频繁调用接收rtcm消息
                tp_SleepTask(10);
            }
        } else {
            printf("recv rtcm faild ,ret= %derrorcode: %d\n",rtcmRet, tp_get_errorcode(rtcmRet));
            tp_SleepTask(1000);
        }
        // The sleep time needs to be adjusted according to the actual situation
        tp_SleepTask(1000);
    }
}

int main(int argc, char *argv[]) {

    const char *user = "username";
    const char *pwd = "passwd";
    const char *mountpoint = "RTCM33GRCEJ";
    const char *devId = "NULL";
    int freq = 1;
    int authtype = TP_CLOUD_CHIP_NTRIP;
    int block_time = freq + 2;

    int baudrate = 460800;
    const char *serial = "/dev/ttyUSB0";
    pthread_t rtcm_thread = 0;
    int ret = 0;
```

```
    /* 初始化串口 */
    if (init_tty((const UINT8 *)serial, baudrate) < 0) {
        printf("tty_set_device failed\n");
    }

    // 获取打印SDK版本信息
    St_sdk_response_t response;
    tp_sdk_query(GET_SDK_VERSION, &response, NULL);
    printf("true-point sdk version is %s\n", response.uResult.com_result.buf);

    // 设置日志打印方法。不设置不打印。
    tp_set_log_cb(log_callback);

    // 设置socket回调函数，用于在创建连接时进行自定义操作。
    tp_set_socket_cb(set_socket_callback);

    // 设置配置文件路径。这里传入配置文件路径，SDK会在初始化时内部自动加载配置文件。如果不需要加载配置文件，可以不调用此函数。
    ret = tp_set_config_path((const INT8*)"");
    printf("invok tp_set_config_path return %d, errcode %d\n", ret, tp_get_errorcode(ret));
    if (ret != 0) {
        goto CLEAN2;
    }

    /* 初始化。必须判断是否成功，失败则应报错不能继续执行*/
    ret = tp_sdk_init((const INT8*)user, (const INT8*)pwd, (const INT8*)devId, (const INT8*)mountpoint, freq, authtype);
    printf("invok tp_sdk_init return %d, errcode %d\n", ret, tp_get_errorcode(ret));
    if (ret != 0) {
        goto CLEAN2;
    }

    // 更新配置
    TPConfigFull_t tpConfig;
    //这里必须得全部设置为-1，否则后续更新时无法识别哪些是更新的，哪些是原有不需要修改的值。导致更新错误
    memset(&tpConfig, -1, sizeof(tpConfig));
    tpConfig.block_time = block_time;
    tpConfig.coordsys = 1;
    tpConfig.send_gga_with_retry = 1;
    tpConfig.tli_protocol  = TLI_TCP;
    // 关闭日志打印。5表示关闭所有日志打印
    tpConfig.loglevel  = LOG_LEVEL_NONE;
    // 更新配置。更新失败也应进行报错处理。
    ret = tp_sdk_config(&tpConfig);
    if (ret != 0) {
        goto CLEAN2;
```

```
}

// 获取配置，打印查看配置是否设置成功。
// log_info("tpConfig: %s\n", tpconfig_to_string(tp_get_current_config(&tpConfig)));

/* 使用默认配置的域名和端口建立连接。 */
ret = tp_connect_service(NULL, 0);
printf("invok tp_connect_service return %d, errcode %d\n", ret, tp_get_errorcode(ret));
if (ret != 0) {
    goto CLEAN1;
}

// 获取tcp网络文件描述符。参数为保留值，固定传0即可
int fd = tp_get_network_fd(0);
printf("tlint fd is %d\n", fd);

/* create thread to receive the correction data*/
while (rtcm_thread == 0) {
    ret = pthread_create(&rtcm_thread, NULL, (void *)&handle_rtcm_recv, NULL);
    printf("invok rtcm thread return %d\n", ret);
    if (ret != 0) {
        tp_SleepTask(100);
    }
}

// 主线程中持续发送gga消息
while (1) {
    char gga[128] = {0};

    int ggaLen = ttyread((UINT8 *)gga, sizeof(gga));
    if (ggaLen < 0) {
        printf("read serial failed");
        break;
    }
    printf("ggalen is %d,gga is: %s\n", ggaLen, gga);
    ret = tp_send_gga((const INT8*)gga, ggaLen);
    printf("invode tp_send_gga return %d, errcode %d, gga is %s\n", ret, tp_get_errorcode(ret), gga);
    if (ret != 0) {
        // 如果是一般错误可以继续执行，如果遇到账号过期之类的无法继续执行的错误应报警。
        printf("send_gga fail");
    }
    tp_SleepTask(1000);
}

// 等待全部接收完
tp_SleepTask(freq * 1000);
// 结束接收进程
isstop = 1;
```

```
    tp_SleepTask(100);
    pthread_join(rtcm_thread, NULL);

    /* stop and cleanup SDK. */
CLEAN1:
    tp_sdk_stop();
CLEAN2:
    tp_sdk_cleanup();
}
```

8 联系与帮助

1. 联系工作人员：任何产品服务咨询、问题解答、服务报障、意见建议等，可直接联系负责的客户经理。
2. 提交在线工单：登录真点用户控制台，提交在线工单服务（7x24 小时快速处理）。
   - 国内服务用户控制台： https://console.true-point.com
   - 海外服务用户控制台： https://enconsole.true-point.com
3. 发送邮件咨询：产品服务咨询、意见建议等，可发送邮件至 info@true-point.com。

感知精准时空，赋能智慧世界

真点科技（北京）有限公司
TruePoint Technology Inc.

北京市海淀区丰贤东路 7 号北斗星通大厦北楼二层

www.true-point.com

info@true-point.com

86-10-69939988